



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

Unidad 2. Práctica 3. Logs - Parte 2.

1. Datos generales

Campo	Valor
Proyecto	VulnerableApp, rama secure
Framework	ASP.NET Core MVC sobre .NET 10
Biblioteca de logging	Serilog
Serilog.AspNetCore	10.0.0
Serilog.Sinks.Console	6.1.1
Serilog.Sinks.File	7.0.0
Serilog.Sinks.Seq	9.1.0
Serilog.Enrichers.Environment	3.0.1
Serilog.Enrichers.Thread	4.0.0
Servidor de agregación	Seq, imagen datalust/seq:latest en Docker
Endpoint de ingesta	http://localhost:5341
Interfaz web de consulta	http://localhost:8081
Controladores instrumentados	HomeController, SearchController, AuthController, CommentController, ApiController
Ruta de persistencia en disco	VulnerableApp/Logs/log-YYYYMMDD.txt
Política de rotación	Diaria (RollingInterval.Day)

2. Objetivo y alcance

Implementar una estrategia de registro de eventos estructurado sobre una aplicación ASP.NET Core, sustituyendo el proveedor de logging predeterminado del framework por Serilog, y centralizar la totalidad de los eventos generados en un servidor Seq desplegado en contenedor, de modo que resulte posible consultar, filtrar y correlacionar la actividad de la aplicación mediante propiedades y no únicamente mediante búsqueda textual.

El alcance comprende la instrumentación de los cinco controladores de la aplicación con registro de entrada, salida, duración, identidad del solicitante, dirección de origen, parámetros de la petición, advertencias de seguridad, errores y excepciones, así como la verificación empírica —con tráfico real— de que los controles implementados operan según lo previsto y de que ningún dato sensible alcanza los registros.

3. Marco conceptual

3.1 Logging estructurado frente a logging textual

El registro tradicional produce cadenas de texto planas en las que los datos variables quedan interpolados dentro del mensaje. Su consulta posterior depende de búsquedas por coincidencia de subcadenas, técnica frágil ante cambios de redacción y que no permite agregaciones ni filtros sobre los valores individuales.

El logging estructurado, en cambio, conserva el mensaje como **plantilla** y los valores como **propiedades nombradas e independientes**. La instrucción `_logger.LogInformation("Inicio Auth.Login Usuario:{User} IP:{IP}", username, ip)` no produce una cadena ya resuelta, sino un evento con la plantilla y dos propiedades tipadas, `User` e `IP`. Esta representación permite que el destino de almacenamiento indexe cada propiedad por separado y habilita consultas del tipo «todos los eventos donde `User` sea un valor determinado», imposibles de expresar de forma confiable sobre texto plano.

Esta distinción es la que justifica la introducción de Serilog frente al proveedor predeterminado de `Microsoft.Extensions.Logging`, y la que otorga sentido a la incorporación de un servidor de agregación como Seq: sin estructura, la centralización se reduciría a concentrar archivos de texto en un mismo lugar.

3.2 Sinks y enrichers

Dos abstracciones organizan la arquitectura de Serilog y conviene precisarlas antes de describir la configuración.

Un **sink** es un destino de escritura. Serilog admite múltiples sinks simultáneos sobre un mismo evento, y cada uno puede aplicar su propio nivel mínimo y su propio formato de salida. La escritura ocurre en paralelo desde el punto de vista lógico, de modo que un mismo evento puede quedar simultáneamente impreso en consola, persistido en disco y transmitido por red.

Un **enricher** es un componente que agrega propiedades adicionales a todo evento que atraviesa el pipeline, con independencia del punto del código donde se haya originado. Su valor consiste en incorporar contexto que el desarrollador no necesita

repetir en cada llamada: el nombre del host, el identificador del hilo de ejecución o el nombre de la aplicación se adjuntan de forma transversal y automática.

4. Desarrollo

4.1 Incorporación de paquetes

Se agregaron al proyecto los paquetes correspondientes al núcleo de Serilog para ASP.NET Core, a los tres sinks requeridos y a los dos enrichers adicionales:

```
dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.Console
dotnet add package Serilog.Sinks.File
dotnet add package Serilog.Sinks.Seq
dotnet add package Serilog.Enrichers.Environment
dotnet add package Serilog.Enrichers.Thread
```

4.2 Despliegue de Seq en contenedor

Seq se desplegó como contenedor con reinicio automático, exponiendo el puerto de ingesta y el de la interfaz web:

```
docker run --name seq -d --restart unless-stopped ^
-e ACCEPT_EULA=Y ^
-e SEQ_FIRSTRUN_NOAUTHENTICATION=True ^
-p 5341:80 -p 8081:80 ^
datalust/seq:latest
```

Hallazgo de configuración. El comando indicado en la guía original de la práctica —idéntico al anterior salvo por la ausencia de la variable `SEQ_FIRSTRUN_NOAUTHENTICATION`— **no logra arrancar el contenedor** en la versión actual de la imagen. Desde una revisión reciente, Seq exige que en el primer arranque se defina una contraseña de administrador mediante `SEQ_FIRSTRUN_ADMINPASSWORD` o bien se desactive explícitamente la autenticación mediante `SEQ_FIRSTRUN_NOAUTHENTICATION`. Sin ninguna de las dos, el proceso interno aborta durante la inicialización con el error `No default admin password was supplied`, y la directiva `--restart unless-stopped` provoca que el contenedor entre en un ciclo indefinido de reinicio, permaneciendo en estado `Restarting` sin llegar nunca a atender peticiones.

Este comportamiento constituye una decisión deliberada del fabricante orientada a evitar despliegues sin autenticación por omisión, y resulta coherente con el principio de configuración segura por defecto. La desactivación aplicada en esta práctica es

admisible únicamente por tratarse de un entorno local de laboratorio sin exposición a red; en cualquier despliegue accesible desde otros equipos debería definirse la contraseña de administrador, dado que Seq concentra información detallada sobre la actividad interna de la aplicación.

4.3 Configuración del punto de entrada

Se adoptó el patrón de **bootstrap logger** recomendado por la documentación oficial de Serilog, en lugar de la configuración en línea propuesta en la guía. El patrón consiste en crear un logger mínimo antes de construir el host, sustituirlo por la configuración definitiva una vez que el contenedor de dependencias está disponible, y envolver toda la ejecución en un bloque de manejo de excepciones.

```
csharp
```

```
using Microsoft.EntityFrameworkCore;
```

```
using Microsoft.Extensions.Hosting;
```

```
using Serilog;
```

```
using VulnerableApp.Data;
```

```
Log.Logger = new LoggerConfiguration()  
    .WriteTo.Console()  
    .CreateBootstrapLogger();
```

```
Log.Information("Iniciando VulnerableApp...");
```

```
try
```

```
{
```

```
    var builder = WebApplication.CreateBuilder(args);
```

```
    builder.Host.UseSerilog((context, services, configuration) => configuration  
        .ReadFrom.Configuration(context.Configuration)  
        .ReadFrom.Services(services)  
        .Enrich.FromLogContext());
```

```
    builder.Services.AddControllersWithViews();
```

```

builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnecti
on")));

builder.Services.AddSession();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseSerilogRequestLogging();
app.UseRouting();
app.UseSession();
app.UseAuthorization();
app.MapStaticAssets();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}").WithStaticAssets();

app.Run();
}
catch (Exception ex) when (ex is not HostAbortedException)
{
    Log.Fatal(ex, "VulnerableApp termino inesperadamente durante el arranque");
}

```

```
finally
{
    Log.CloseAndFlush();
}
```

Tres decisiones de esta configuración merecen justificación explícita.

Bootstrap logger. Una configuración de logging construida únicamente después de `builder.Build()` deja sin cobertura toda la fase previa de arranque. Cualquier excepción producida durante el registro de servicios, la lectura de configuración o la resolución de la cadena de conexión ocurriría antes de que exista un logger, y se perdería sin registro alguno. El bootstrap logger cubre esa ventana y es reemplazado de forma transparente por la configuración definitiva una vez que esta queda disponible.

Posición de `UseSerilogRequestLogging()`. Este middleware condensa en un único evento resumido lo que el registro predeterminado de ASP.NET Core distribuye en varias líneas por petición, incorporando método, ruta, código de estado y duración. Su ubicación temprana en el pipeline resulta necesaria para que la medición de duración comprenda el tiempo consumido por los middlewares posteriores; situarlo al final produciría mediciones sistemáticamente inferiores a la latencia real percibida por el cliente.

Exclusión de `HostAbortedException`. El filtro `when (ex is not HostAbortedException)` responde a una condición detectada durante la propia práctica. Las herramientas de línea de comandos de Entity Framework Core construyen el host de la aplicación para inspeccionar los servicios registrados y, acto seguido, lo abortan de forma deliberada mediante esta excepción, que constituye por tanto un mecanismo de control de flujo previsto y no una falla. Sin la exclusión, cada ejecución de `dotnet ef database update` producía un evento de nivel Fatal en los registros, contaminando el historial con falsos positivos de máxima severidad. El detalle de este hallazgo se documenta en la sección 7.

`Log.CloseAndFlush()` en el bloque `finally`. Los sinks de archivo y de red operan con escritura almacenada en búfer por razones de rendimiento. Sin el vaciado explícito al terminar, los últimos eventos generados —precisamente los más relevantes cuando la aplicación finaliza de forma anómala— podrían no alcanzar su destino.

4.4 Configuración externa del comportamiento del logger

La totalidad del comportamiento de Serilog se declaró en `appsettings.json`, quedando el código libre de configuración de destinos y niveles:

json

```
"Serilog": {  
  "MinimumLevel": {  
    "Default": "Information",  
    "Override": {  
      "Microsoft": "Warning",  
      "Microsoft.AspNetCore": "Warning",  
      "Microsoft.EntityFrameworkCore": "Warning"  
    }  
  },  
  "WriteTo": [  
    { "Name": "Console" },  
    { "Name": "File", "Args": { "path": "Logs/log-.txt", "rollingInterval": "Day" } },  
    { "Name": "Seq", "Args": { "serverUrl": "http://localhost:5341" } }  
  ],  
  "Enrich": [ "FromLogContext", "WithMachineName", "WithThreadId" ],  
  "Properties": { "Application": "VulnerableApp" }  
}
```

El archivo appsettings.Development.json sobrescribe únicamente los niveles, dejando intactos sinks y enrichers:

json

```
"Serilog": {  
  "MinimumLevel": {  
    "Default": "Debug",  
    "Override": {  
      "Microsoft": "Information",  
      "Microsoft.AspNetCore": "Warning",  
      "Microsoft.EntityFrameworkCore": "Information"  
    }  
  }  
}
```


}

Los **overrides por espacio de nombres** cumplen una función determinante para la utilidad del registro. Sin ellos, el nivel Information aplicado de forma global provocaría que el framework y Entity Framework Core emitieran un volumen elevado de eventos —cada consulta SQL generada, cada resolución de ruta— que sepultaría los eventos propios de la aplicación. Elevar el umbral de esos espacios de nombres a Warning conserva sus advertencias y errores, que sí resultan pertinentes, mientras suprime el ruido informativo. La configuración de desarrollo relaja selectivamente el umbral de Entity Framework Core a Information, nivel en el que sí registra las consultas ejecutadas, lo cual resulta deseable durante la depuración pero no en operación normal.

La propiedad Properties.Application adjunta un identificador constante a todo evento emitido. En un servidor Seq que recibe registros de varias aplicaciones, esta propiedad es la que permite segmentar el flujo por origen.

```
VulnerableApp > Program.cs
10 Log.Logger = new LoggerConfiguration()
11     .WriteTo.Console()
12     .CreateBootstrapLogger();
13
14 Log.Information("Iniciando VulnerableApp...");
15
16 try
17 {
18     var builder = WebApplication.CreateBuilder(args);
19
20     // Reemplaza el logger de bootstrap por el pipeline completo definido
21     // en appsettings.json (Serilog: MinimumLevel, WriteTo, Enrich).
22     builder.Host.UseSerilog((context, services, configuration) => configuration
23         .ReadFrom.Configuration(context.Configuration)
24         .ReadFrom.Services(services)
25         .Enrich.FromLogContext());
26
27     // Add services to the container.
28     builder.Services.AddControllersWithViews();
29
30     builder.Services.AddDbContext<AppDbContext>(options =>
31         options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
32
33     builder.Services.AddSession();
34
35     var app = builder.Build();
36
37     // Configure the HTTP request pipeline.
38     if (!app.Environment.IsDevelopment())
39     {
40         // El HSTS es una cabecera de seguridad de transporte, no forma parte
41         // del manejo de excepciones: se conserva para producción.
42         app.UseHsts();
43     }
44 }
```

4.5 Instrumentación de los controladores

Los cinco controladores requeridos reciben `ILogger<T>` mediante inyección de dependencias y registran, en cada acción:

- **Entrada a la acción**, en nivel `Information`, incluyendo identidad del solicitante y dirección IP de origen.
- **Salida de la acción**, en nivel `Information`, incorporando la duración en milisegundos medida con `Stopwatch`.
- **Parámetros de la petición**, con exclusión deliberada de todo campo de credencial.
- **Advertencias**, en nivel `Warning`: intento de autenticación fallido, acceso sin sesión establecida, acceso a recurso ajeno y envío de comentario vacío.
- **Errores**, en nivel `Error`, mediante `LogError(ex, ...)` con la excepción como primer argumento, de modo que `Serilog` capture la traza de pila como propiedad estructurada y no como texto concatenado en el mensaje.
- **Eventos de autenticación** con el prefijo convencional `AuthEvent`: seguido del tipo de evento: `LoginSuccess`, `LoginFailed`, `Logout`, `AccesoNoAutenticado` y `AccesoNoAutorizado`.

La adopción de un prefijo común para los eventos de autenticación constituye una decisión de diseño orientada a la consulta posterior. Permite recuperar la totalidad de la actividad relacionada con el control de acceso mediante un único filtro, con independencia del controlador que la haya originado, y establece una convención que un equipo puede extender de forma consistente.

Fragmento representativo del registro de un intento fallido de autenticación:

```
csharp
```

```
_logger.LogInformation("Inicio Auth.Login (POST) Usuario:{User} IP:{IP}",  
username, ip);
```

```
...
```

```
_logger.LogWarning("AuthEvent:LoginFailed Usuario:{User} IP:{IP}  
Motivo:CredencialesInvalidas",  
username, ip);
```

Obsérvese que el mensaje registra el nombre de usuario y el motivo del rechazo, pero **en ningún caso la contraseña suministrada**. El nombre de usuario es necesario para detectar patrones de ataque —intentos repetidos contra una misma cuenta o contra cuentas distintas desde una misma dirección—, mientras que la

contraseña carece por completo de utilidad diagnóstica y su registro constituiría una exposición de credenciales en texto plano dentro de un archivo que, por su naturaleza, suele replicarse hacia sistemas de respaldo y de agregación con controles de acceso más laxos que los de la base de datos original.

5. Retos resueltos

5.1 Función de cada sink

Sink	Función técnica	Justificación de su inclusión
Console	Escribe cada evento en la salida estándar del proceso, durante el desarrollo local. Es el primer con formato legible y coloreado por nivel.	Proporciona retroalimentación inmediata al ejecutar la aplicación y no depende de ningún servicio externo.
File	Persiste los eventos en archivos de texto en disco, aplicando rotación diaria conforme al patrón log-YYYYMMDD.txt.	Ofrece un historial que sobrevive al reinicio del proceso y a la indisponibilidad del servidor de agregación. Constituye el respaldo de última instancia y la base de la auditoría.
Seq	Transmite los eventos en formato JSON estructurado (CLEF) al servidor mediante HTTP, con envío por lotes.	Habilita búsqueda, filtrado y correlación por propiedades individuales, así como agregaciones y alertas, capacidades inalcanzables mediante búsqueda textual sobre archivos planos.

La combinación responde a una lógica de redundancia deliberada. Los tres sinks presentan modos de falla distintos: la consola desaparece al cerrar el proceso, el archivo depende de la disponibilidad del disco y de la política de rotación, y el sink de red depende de que el servidor esté operativo. Ante la caída de Seq, el sink correspondiente descarta o encola los eventos según su configuración, pero los sinks de consola y archivo continúan operando sin interrupción, de modo que ningún evento se pierde por completo.

5.2 Control del nivel mínimo desde configuración

El requisito se satisfizo mediante la instrucción `configuration.ReadFrom.Configuration(context.Configuration)` dentro de `UseSerilog`, que delega la totalidad de la configuración del logger al sistema de configuración de ASP.NET Core. Este sistema resuelve el valor efectivo aplicando la cadena de proveedores en orden de precedencia: `appsettings.json` como base,

appsettings.{Environment}.json como sobrescritura según el valor de ASPNETCORE_ENVIRONMENT, y las variables de entorno por encima de ambos.

La consecuencia práctica es que el nivel de detalle del registro puede modificarse en un despliegue sin recompilar ni redistribuir la aplicación, capacidad relevante en operación real: ante un incidente en producción, elevar temporalmente el detalle a Debug constituye una herramienta diagnóstica de primer orden. La verificación empírica de este comportamiento se documenta en la sección 6.7.

5.3 Enrichers adicionales y su justificación

Más allá de FromLogContext, requerido por la práctica, se incorporaron dos enrichers adicionales.

WithMachineName adjunta el nombre del host que generó el evento, LAPTOP-1M1UUQDQ en este entorno. Su justificación aparece en despliegues con más de una instancia —contenedores replicados, máquinas virtuales tras un balanceador de carga—, donde permite determinar en qué nodo se originó cada evento sin necesidad de inspeccionar los registros de cada servidor por separado. Resulta particularmente valioso ante fallas que se manifiestan en una sola instancia, escenario en el que la propiedad convierte un problema difícil de reproducir en una consulta directa.

WithThreadId adjunta el identificador del hilo que procesó la petición. ASP.NET Core atiende peticiones concurrentes sobre un pool de hilos, de modo que los eventos de varias peticiones simultáneas quedan entrelazados en el flujo de salida. La marca temporal, aun con resolución de milisegundos, no siempre permite separar dos ejecuciones concurrentes, mientras que el identificador de hilo constituye un discriminante inequívoco para reconstruir la secuencia correspondiente a una misma ejecución.

Corresponde precisar una limitación de este último enricher en modelos asíncronos: dado que una operación async puede reanudarse en un hilo distinto al que la inició, el identificador no permanece constante a lo largo de toda una petición. Para correlación estricta extremo a extremo, la construcción adecuada es un identificador de correlación propagado mediante LogContext, mecanismo que FromLogContext habilita y que el registro automático de peticiones de Serilog aprovecha internamente.

6. Pruebas ejecutadas y resultados

6.1 Tráfico de prueba generado

Los cinco controladores se ejercitaron mediante curl, cubriendo la página principal, búsqueda con y sin término, autenticación fallida y exitosa, acceso al panel con y sin sesión, alta de comentario válido y vacío, y los tres escenarios de la API: respuesta 401 sin autenticación, respuesta 200 sobre el recurso propio y respuesta 403 ante el intento de acceso al recurso de otro usuario.

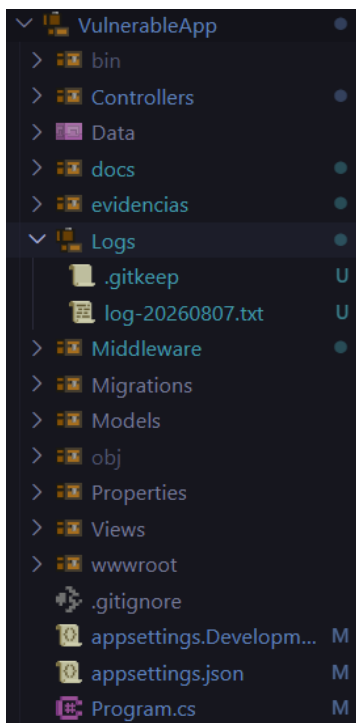
Esta cobertura fue diseñada de forma deliberada para ejercitar **rutas de éxito y rutas de rechazo en igual proporción**. Un conjunto de pruebas limitado a peticiones válidas produciría un registro compuesto exclusivamente por eventos de nivel Information, dejando sin verificar precisamente la instrumentación de advertencias y errores, que es la de mayor valor operativo.

6.2 Evidencia del sink de consola

La ejecución de la aplicación produce en la terminal el flujo de eventos con formato legible y prefijos de nivel abreviados, intercalando eventos propios de la aplicación y eventos automáticos del middleware de registro de peticiones.

6.3 Evidencia del sink de archivo

El sink de archivo crea el directorio Logs durante la primera escritura y genera un archivo por día conforme al patrón de rotación configurado.



```
VulnerableApp > Logs > log-20260807.txt
1 2026-08-07 04:06:54.578 -06:00 [INF] [cid:] Now listening on: http://localhost:5277
2 2026-08-07 04:06:54.600 -06:00 [INF] [cid:] Application started. Press Ctrl+C to shut down.
3 2026-08-07 04:06:54.602 -06:00 [INF] [cid:] Hosting environment: Development
4 2026-08-07 04:06:54.602 -06:00 [INF] [cid:] Content root path: C:\Users\victo\Documents\practicas\VulnerableApp
5 2026-08-07 04:07:14.747 -06:00 [WRN] [cid:589679fd-3f4c-49bd-968f-39966dc3dbb5] Failed to determine the https port for redirect.
6 2026-08-07 04:07:14.846 -06:00 [INF] [cid:589679fd-3f4c-49bd-968f-39966dc3dbb5] Inicio Home.Index Usuario:anonimo IP:::1
7 2026-08-07 04:07:14.847 -06:00 [INF] [cid:589679fd-3f4c-49bd-968f-39966dc3dbb5] Fin Home.Index Usuario:anonimo DuracionMs:5
8 2026-08-07 04:07:14.966 -06:00 [INF] [cid:589679fd-3f4c-49bd-968f-39966dc3dbb5] HTTP GET / respondio 200 en 216.0239 ms
9 2026-08-07 04:07:15.249 -06:00 [INF] [cid:b15c2e45-34fd-42d4-878b-1fdb242f23a1] Inicio Search.Index Usuario:anonimo IP:::1 Parametros:{"search":"_CONTR
10 2026-08-07 04:07:15.250 -06:00 [ERR] [cid:b15c2e45-34fd-42d4-878b-1fdb242f23a1] Excepcion controlada capturada en Search.Index Usuario:anonimo IP:::1
11 System.ArgumentException: Termino de busqueda de prueba invalido (excepcion controlada)
12 at VulnerableApp.Controllers.SearchController.Index(String search) in C:\Users\victo\Documents\practicas\VulnerableApp\Controllers\SearchController.cs:line 12
13 2026-08-07 04:07:15.346 -06:00 [INF] [cid:b15c2e45-34fd-42d4-878b-1fdb242f23a1] HTTP GET /Search/Index respondio 200 en 281.3204 ms
14 2026-08-07 04:07:15.515 -06:00 [INF] [cid:1fcbd2ba-b8c0-422c-85bb-69990d6d24a8] Inicio Auth.Login (POST) Usuario:admin IP:::1
15 2026-08-07 04:07:15.926 -06:00 [WRN] [cid:1fcbd2ba-b8c0-422c-85bb-69990d6d24a8] No store type was specified for the decimal property 'Balance' on entity
16 2026-08-07 04:07:16.928 -06:00 [INF] [cid:1fcbd2ba-b8c0-422c-85bb-69990d6d24a8] Executed DbCommand (111ms) [Parameters=[@username=?' (Size = 4000)], Co
17 SELECT TOP(1) [u].[Id], [u].[Balance], [u].[CreatedAt], [u].[Email], [u].[PasswordHash], [u].[Username]
18 FROM [Users] AS [u]
19 WHERE [u].[Username] = @username
20 2026-08-07 04:07:16.978 -06:00 [DBG] [cid:1fcbd2ba-b8c0-422c-85bb-69990d6d24a8] Verificando hash BCrypt para Usuario:admin (password nunca se registra en
21 2026-08-07 04:07:17.195 -06:00 [INF] [cid:1fcbd2ba-b8c0-422c-85bb-69990d6d24a8] AuthEvent:LoginSuccess Usuario:admin IP:::1 DuracionMs:1679
22 2026-08-07 04:07:17.200 -06:00 [INF] [cid:1fcbd2ba-b8c0-422c-85bb-69990d6d24a8] HTTP POST /Auth/Login respondio 302 en 1707.4801 ms
23 2026-08-07 04:07:17.439 -06:00 [INF] [cid:3efc8d2a-8f1a-49b3-ad6b-bfee0968f534] Inicio Api.GetUser IP:::1 Parametros:{"id":999}
24 2026-08-07 04:07:17.440 -06:00 [ERR] [cid:3efc8d2a-8f1a-49b3-ad6b-bfee0968f534] Excepcion controlada capturada en Api.GetUser IP:::1
25 System.ArgumentOutOfRangeException: Id de prueba invalido (excepcion controlada) (Parameter 'id')
26 at VulnerableApp.Controllers.ApiController.GetUser(Int32 id) in C:\Users\victo\Documents\practicas\VulnerableApp\Controllers\ApiController.cs:line 45
27 2026-08-07 04:07:17.499 -06:00 [WRN] [cid:3efc8d2a-8f1a-49b3-ad6b-bfee0968f534] HTTP GET /api/user/-999 respondio 400 en 89.7794 ms
28 2026-08-07 04:07:17.637 -06:00 [INF] [cid:6b03ac3a-ca68-49d1-9a88-2d9c7f063bca] Inicio Comment.AddComment Usuario:admin IP:::1 Parametros:{"comment":"_U
29 2026-08-07 04:07:17.641 -06:00 [ERR] [cid:6b03ac3a-ca68-49d1-9a88-2d9c7f063bca] Unhandled exception. Metodo:POST Ruta:/Comment/AddComment CorrelationId:6
30 System.InvalidOperationException: Excepcion de prueba NO controlada (Comment.AddComment)
31 at VulnerableApp.Controllers.CommentController.AddComment(String comment) in C:\Users\victo\Documents\practicas\VulnerableApp\Controllers\CommentController.cs:line 12
32 at lambda_method77(closure, Object, Object[])
33 at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.SyncActionResultExecutor.Execute(ActionContext actionContext, IActionResultTypeMapper
34 at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeActionMethodAsync()
35 at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State& next, Scope& scope, Object& state, Boolean& isCompleted)
36 at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeNextActionFilterAsync()
37 --- End of stack trace from previous location ---
```

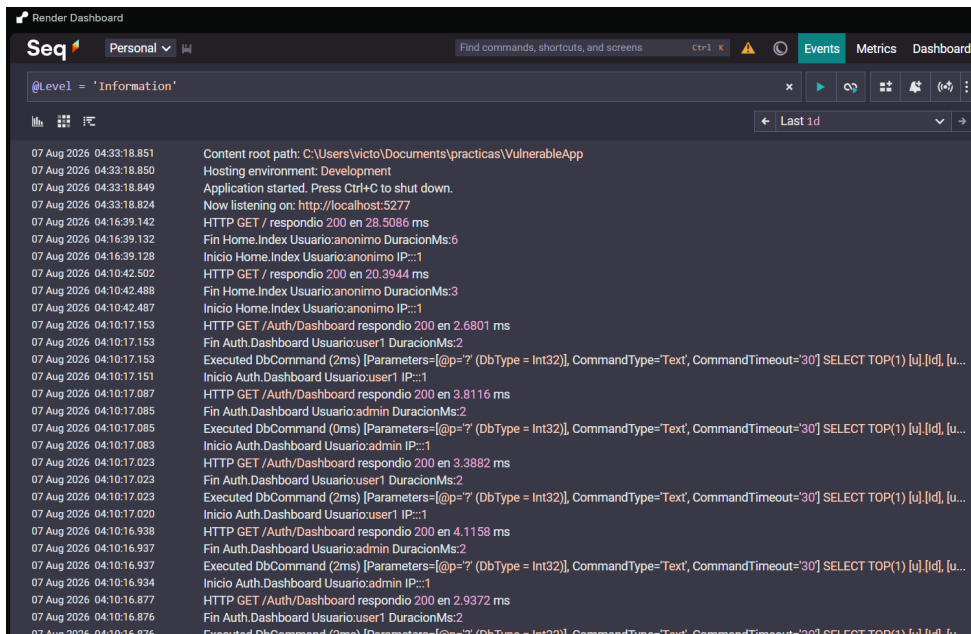
Distribución de eventos por nivel en el archivo correspondiente a la sesión de pruebas:

Nivel	Cantidad
Debug	8
Information	64
Warning	6
Error	5

Los cinco eventos de nivel Error corresponden a la corrida dedicada a provocar una falla real, descrita en la sección 6.6.

6.4 Evidencia del sink de red y de la agregación en Seq

La interfaz web de Seq presenta el flujo de eventos recibidos, con coloreado por nivel y actualización en tiempo real.



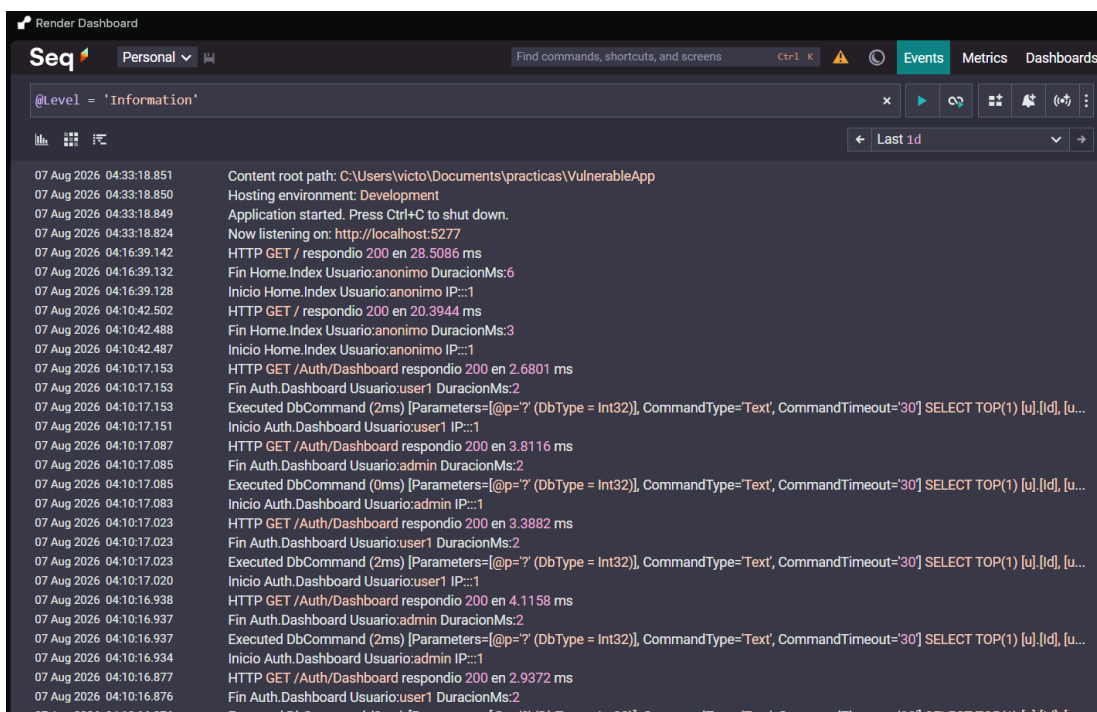
6.5 Consulta filtrada por controlador y por tipo de evento

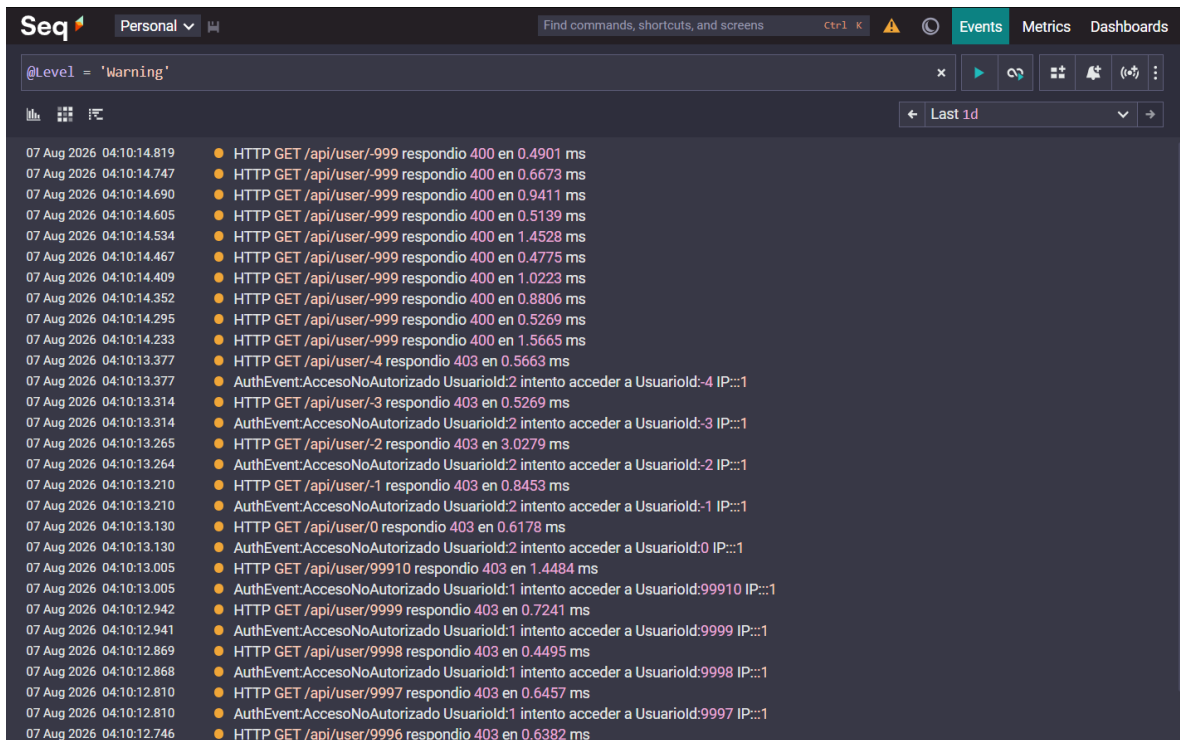
La capacidad de filtrado constituye la justificación principal de la incorporación de Seq. Las consultas se expresan sobre las propiedades del evento y no sobre el texto renderizado:

@Message like '%Search.Index%'

@Message like '%AuthEvent%'

@Level = 'Warning'





Los eventos recuperados en esta última consulta poseen valor de seguridad directo: constituyen exactamente el conjunto que un monitoreo operativo vigilaría para detectar intentos de acceso no autorizado, y su concentración bajo un prefijo común permite construir sobre ellos una alerta automática.

6.6 Prueba de generación de error real

Se provocó una falla de conectividad con el motor de base de datos, redirigiendo la cadena de conexión hacia una instancia inexistente, y se emitió una petición al controlador de búsqueda. La falla quedó capturada de forma íntegra en los tres sinks:

```
[ERR] An error occurred using the connection to database 'VulnerableDb' on server  
      '(localdb)\instancia_inexistente_test'.
```

```
[ERR] An exception occurred while iterating over the results of a query for context  
type
```

```
      'VulnerableApp.Data.AppDbContext'.
```

```
[ERR] Error en Search.Index Usuario:anonimo IP:::1 Parametros:{"search":"admin"}
```

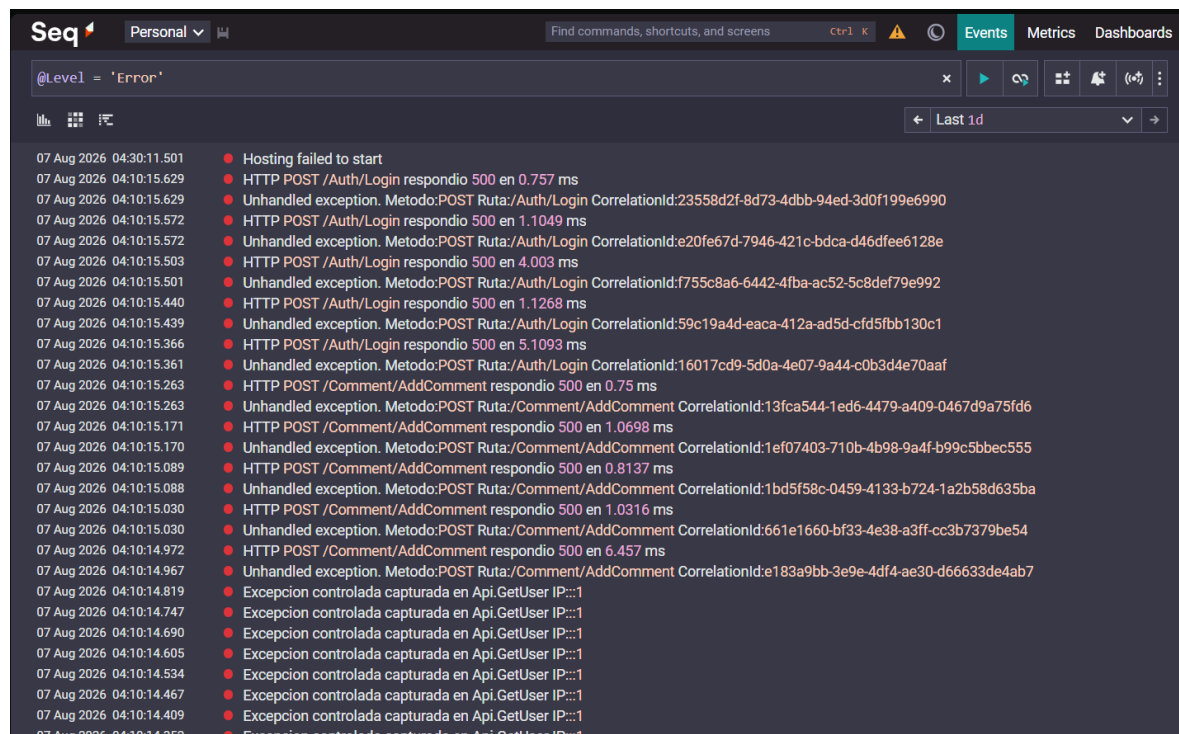
```
[ERR] HTTP GET /Search/Index responded 500 in 3700.1909 ms
```

```
[ERR] An unhandled exception has occurred while executing the request.
```

La secuencia resulta ilustrativa de cómo se estratifica el registro en una arquitectura por capas. La tercera línea corresponde al LogError explícito incorporado en el

controlador, único evento que aporta contexto de negocio —qué usuario, desde qué dirección y con qué parámetros—; las restantes son generadas automáticamente por Entity Framework Core, por el middleware de registro de peticiones y por el manejador global de excepciones. La lectura conjunta permite reconstruir la falla desde el síntoma observable por el cliente hasta su causa técnica en la capa de datos.

Merece atención la duración registrada, cercana a los 3,7 segundos: no corresponde al tiempo de procesamiento sino al agotamiento del temporizador de conexión del proveedor de datos. Este dato, que ningún registro no instrumentado habría capturado, permite distinguir una falla por indisponibilidad del servidor de una falla por credenciales incorrectas o por consulta malformada, que se manifestarían de inmediato.



6.7 Verificación empírica del control de nivel desde configuración

Para comprobar de forma reproducible que el nivel mínimo declarado en configuración gobierna efectivamente el registro, se ejecutó **la misma secuencia de tres peticiones** —búsqueda, autenticación y consulta a la API— en dos ocasiones, modificando exclusivamente la variable ASPNETCORE_ENVIRONMENT y sin alterar una sola línea de código.

Entorno	Nivel efectivo	mínimo Eventos Debug	Eventos Information	Eventos Warning
Production	Information	0	16	2
Development	Debug	3	16	2

Los tres eventos que aparecen únicamente bajo el entorno de desarrollo son:

[DBG] Ejecutando consulta parametrizada Users.Where(Username contains admin)

[DBG] Verificando hash BCrypt para Usuario:admin (password nunca se registra en logs)

[DBG] Comparando Usuariold solicitado:1 con Usuariold de sesion:1

El resultado demuestra que el filtrado opera conforme a lo previsto, y su diseño experimental permite atribuir la diferencia exclusivamente a la variable modificada: la invariancia de los conteos de Information y Warning entre ambas corridas confirma que la secuencia de peticiones fue idéntica y que el único factor alterado fue el umbral de registro.

El contenido de estos tres eventos resulta además significativo desde la perspectiva de la materia. Los tres documentan la ejecución de un control de seguridad — parametrización de la consulta, verificación del hash de la credencial y comprobación de pertenencia del recurso—, es decir, exactamente las mitigaciones implementadas en la rama secure del proyecto. Su registro en nivel Debug constituye la elección adecuada: aportan valor durante la depuración y la auditoría de la implementación, pero su volumen y su detalle los tornan inapropiados para operación continua.

Nota sobre la evidencia gráfica: esta verificación se documenta mediante los conteos tabulados y los fragmentos de log transcritos, sin captura de pantalla asociada.

6.8 Verificación de ausencia de credenciales en los registros

Se ejecutó una búsqueda insensible a mayúsculas sobre el archivo de registro, empleando tanto la contraseña incorrecta utilizada deliberadamente en la prueba de autenticación fallida como la contraseña correcta empleada en la prueba de autenticación exitosa:

```
bash
```

```
grep -icE "incorrecta|password.:admin" Logs/log-20260807.txt
```

```
# resultado: 0
```

El resultado nulo confirma que **ninguna de las dos credenciales aparece en el registro**, y que únicamente se persiste el nombre de usuario. La prueba verifica que el código del controlador de autenticación no interpola en ningún punto la variable de contraseña dentro de una plantilla de mensaje.

Conviene subrayar el valor metodológico de esta comprobación. La ausencia de credenciales en los registros constituye una propiedad que la revisión visual del código puede fácilmente pasar por alto, dado que basta una única instrucción de registro descuidada, situada en una ruta de error poco frecuentada, para invalidarla. La verificación mediante búsqueda sobre el registro efectivamente producido examina el resultado observable en lugar de la intención del código, y por ello constituye evidencia de naturaleza más fuerte.

6.9 Verificación mediante la API de Seq

Como evidencia complementaria a las capturas de la interfaz, se consultó directamente la API del servidor:

GET <http://localhost:5341/api/events?count=2000>

Resultados agregados:

Métrica	Valor
Eventos totales recibidos	349
Nivel Information	295
Nivel Warning	34
Nivel Error	10
Nivel Debug	8
Nivel Fatal	2

Eventos con propiedad Application = VulnerableApp 349 (100 %)

La cobertura del 100 % en la propiedad Application confirma que la configuración de propiedades globales se aplica de manera efectiva a la totalidad del flujo, sin excepciones ni eventos emitidos por fuera del pipeline configurado.

Precisión sobre la diferencia entre los conteos. Los totales obtenidos de Seq superan a los del archivo consignados en la sección 6.3, y la discrepancia es esperada y explicable. El archivo rota a diario y refleja la sesión de pruebas

concreta, mientras que Seq acumula de forma persistente los eventos de todas las corridas realizadas durante la práctica, incluidas las de desarrollo y depuración previas a las pruebas finales. Ambos valores son correctos dentro de su respectivo alcance temporal, y su comparación ilustra la diferencia funcional entre un registro rotativo local y un repositorio centralizado de historial.

7. Hallazgos de troubleshooting

Durante el desarrollo de la práctica se identificaron y resolvieron dos condiciones no previstas en la guía, que se documentan por su valor formativo.

7.1 Imposibilidad de arranque de Seq con el comando de la guía

Conforme se detalla en la sección 4.2, la imagen actual de Seq rechaza el arranque cuando no se define contraseña de administrador ni se desactiva explícitamente la autenticación. La combinación de este comportamiento con la directiva `--restart unless-stopped` genera un síntoma potencialmente confuso: el contenedor aparece de forma reiterada en la lista de contenedores activos, en estado `Restarting`, lo que puede interpretarse erróneamente como un proceso en curso de inicialización.

El diagnóstico requirió inspeccionar los registros del contenedor con `docker logs seq`, donde el error aparece de forma explícita. La lección procedimental es directa: ante un contenedor que no responde, la primera acción diagnóstica corresponde a la revisión de sus registros y no a la repetición del comando de arranque.

7.2 Eventos de nivel Fatal falsos generados por las herramientas de Entity Framework Core

Los dos eventos de nivel Fatal registrados en Seq corresponden a ejecuciones de `dotnet ef database update` realizadas **antes** de incorporar la cláusula `when (ex is not HostAbortedException)` descrita en la sección 4.3.

El mecanismo subyacente es el siguiente: las herramientas de diseño de Entity Framework Core construyen el host de la aplicación para inspeccionar los servicios registrados y, una vez obtenida esa información, lo abortan de forma deliberada lanzando `HostAbortedException`. Se trata de un mecanismo de control de flujo previsto por el framework, no de una falla. Sin embargo, el bloque `catch` genérico del patrón de bootstrap logger la interceptaba y la registraba con la máxima severidad.

El impacto de este falso positivo no es meramente cosmético. Un registro que contiene eventos de severidad Fatal originados en operaciones rutinarias erosiona la confianza en la clasificación por niveles y, en un entorno con alertas automatizadas, generaría notificaciones espurias que conducen al fenómeno de

fatiga de alertas, en el cual el operador termina por ignorar precisamente las notificaciones que debería atender.

Tras la incorporación del filtro, se verificó que las ejecuciones subsecuentes de comandos de Entity Framework Core no vuelven a producir eventos de este tipo. Ambos hallazgos quedan por tanto documentados como incidencias resueltas y no como pendientes.

8. Tabla resumen de cumplimiento

Métrica	Valor
Controladores instrumentados	5 de 5 (Home, Search, Auth, Comment, Api)
Sinks configurados	3 de 3 (Console, File, Seq)
Enrichers configurados	3 (FromLogContext requerido, más WithMachineName y WithThreadId justificados)
Eventos totales recibidos en Seq	349
Eventos de nivel Information	295
Eventos de nivel Warning	34
Eventos de nivel Error	10
Eventos de nivel Debug	8
Cobertura de la propiedad Application	100 %
Credenciales detectadas en los registros	0
Diferencial de eventos Debug entre entornos	0 en Production frente a 3 en Development, sobre secuencia idéntica
Eventos de autenticación implementados	LoginSuccess, LoginFailed, Logout, AccesoNoAutenticado, AccesoNoAutorizado
Política de rotación de archivo	Diaria, verificada

Métrica

Valor

Hallazgos de troubleshooting
documentados y resueltos 2

9. Conclusiones

Se configuró Serilog con tres sinks y tres enrichers, con el nivel mínimo gobernado íntegramente desde los archivos de configuración y sin dependencia de constantes en el código. Los cinco controladores requeridos quedaron instrumentados con `ILogger<T>`, registrando entrada, salida, duración, identidad, dirección de origen, parámetros, advertencias, errores, eventos de autenticación y excepciones, y la implementación se validó mediante tráfico real y no únicamente por revisión de código.

El aporte central de la práctica reside en la distinción entre registro textual y registro estructurado. La conservación de la plantilla del mensaje y de las propiedades como entidades independientes convierte al registro de un artefacto destinado a la lectura humana en un conjunto de datos consultable, sobre el cual pueden construirse filtros, agregaciones y alertas. La consulta por nivel de advertencia, que recupera de forma inmediata la totalidad de los intentos de acceso rechazados con independencia del controlador que los originó, ilustra con precisión esa diferencia.

Desde la perspectiva de la seguridad, el registro constituye simultáneamente un control y un riesgo. Es un control porque hace observable la actividad del sistema y habilita la detección de patrones de ataque que ninguna revisión de código podría anticipar; es un riesgo porque concentra en un artefacto de menor protección información que la aplicación custodia con cuidado. La verificación explícita de que ninguna credencial alcanza los registros atiende precisamente esa segunda dimensión, y su comprobación empírica sobre el registro efectivamente producido —en lugar de una revisión visual del código— constituye evidencia de naturaleza más robusta.

Los dos hallazgos de troubleshooting aportan una lección adicional. Tanto la exigencia de contraseña de administrador en Seq como el falso positivo de severidad Fatal derivado de las herramientas de Entity Framework Core corresponden a comportamientos no documentados en la guía, resueltos mediante lectura de registros y comprensión del mecanismo subyacente en lugar de ensayo y error. El segundo caso ilustra además una propiedad importante del propio sistema de registro: un evento clasificado con severidad incorrecta degrada el valor de todo el registro, porque la utilidad de la clasificación por niveles depende íntegramente de su fiabilidad.

Reporte de Práctica

SEGG-U2-P3G-3, P3G-4 y P3G-5

Middleware de Logging Global, Monitoreo con Seq y Generación de Evidencias

Todos los valores provienen de una corrida real ejecutada el 2026-08-07 entre las 04:06 y las 04:10 (hora local), verificados cruzando el archivo de log con la API de Seq.

1. Datos generales

Campo	Valor
Proyecto	VulnerableApp, rama secure
Ventana analizada	2026-08-07, 04:06:53 → 04:10:17 (UTC-06:00)
Peticiones generadas	HTTP 764
Eventos de log generados	3365
Archivo analizado	Logs/log-20260807.txt — 640 KB, 5118 líneas
Seq	Interfaz en http://localhost:8081 , API en http://localhost:5341

2. P3G-3 — Middleware de logging global

Se implementaron tres middlewares propios en `VulnerableApp/Middleware/`, registrados en el siguiente orden:

`CorrelationIdMiddleware` → `UseHttpsRedirection` → `RequestLoggingMiddleware`
→ `ExceptionHandlerMiddleware` → `UseRouting` → ...

El orden no es arbitrario. `CorrelationIdMiddleware` va primero para que el identificador exista antes que cualquier otro registro. `RequestLoggingMiddleware` **envuelve** a `ExceptionHandlerMiddleware`, de modo que cuando este último asigna el código 500 ante una excepción no controlada, el bloque `finally` del primero lee un `StatusCode` ya definitivo y nunca registra un código intermedio.

2.1 `CorrelationIdMiddleware`

`csharp`

```

using Serilog.Context;

namespace VulnerableApp.Middleware;

public class CorrelationIdMiddleware
{
    private const string HeaderName = "X-Correlation-ID";
    private readonly RequestDelegate _next;

    public CorrelationIdMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        var correlationId = context.Request.Headers.TryGetValue(HeaderName, out
var existing)
        && !string.IsNullOrEmpty(existing)
        ? existing.ToString()
        : Guid.NewGuid().ToString();

        context.Items[HeaderName] = correlationId;
        context.Response.Headers[HeaderName] = correlationId;

        using (LogContext.PushProperty("CorrelationId", correlationId))
        {
            await _next(context);
        }
    }
}

```

Decisión de diseño. En lugar de pasar el identificador como parámetro en cada llamada de registro, se emplea `LogContext.PushProperty` dentro de un bloque `using`. Serilog adjunta entonces la propiedad a **todo** evento emitido mientras dura

ese ámbito: los de los cinco controladores instrumentados en la práctica anterior, los de Entity Framework Core y los de los otros dos middlewares. La propagación es automática y no requirió modificar una sola línea de los controladores existentes.

El middleware reutiliza el identificador si el cliente lo envía en el encabezado, y solo genera uno nuevo cuando no existe. Este detalle permite que la trazabilidad se extienda a través de varios servicios en una arquitectura distribuida.

2.2 ExceptionHandlingMiddleware

```
csharp
```

```
namespace VulnerableApp.Middleware;
```

```
public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlingMiddleware> _logger;

    public ExceptionHandlingMiddleware(RequestDelegate next,
                                      ILogger<ExceptionHandlingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {

```

```

        var correlationId = context.Items.TryGetValue("X-Correlation-ID", out var
cid)
        ? cid?.ToString() : null;

        _logger.LogError(ex,
            "Unhandled exception. Metodo:{Method} Ruta:{Path}
CorrelationId:{CorrelationId}",
            context.Request.Method, context.Request.Path, correlationId);

        if (!context.Response.HasStarted)
        {
            context.Response.Clear();
            context.Response.StatusCode =
StatusCodes.Status500InternalServerError;
            context.Response.ContentType = "application/json";
            await context.Response.WriteAsJsonAsync(
                new { error = "Ocurrio un error inesperado.", correlationId });
        }
    }
}
}

```

Sustituye al manejador predeterminado de la plantilla. Toda excepción no atendida localmente termina aquí, se registra con su traza completa y su identificador de correlación, y el cliente recibe una respuesta JSON uniforme **sin trazas de pila ni mensajes del motor de base de datos**, evitando la filtración de detalles internos. La devolución del correlationId al cliente cierra el circuito de soporte: el usuario puede reportarlo y el operador localiza el incidente exacto con una sola consulta.

La verificación HasStarted es necesaria porque, una vez enviados los encabezados, el código de estado ya no admite modificación.

2.3 RequestLoggingMiddleware

```

csharp
using System.Diagnostics;

```

```
namespace VulnerableApp.Middleware;

public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestLoggingMiddleware> _logger;

    public RequestLoggingMiddleware(RequestDelegate next,
                                   ILogger<RequestLoggingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var sw = Stopwatch.StartNew();
        try { await _next(context); }
        finally
        {
            sw.Stop();
            var statusCode = context.Response.StatusCode;
            var elapsedMs = sw.Elapsed.TotalMilliseconds;

            if (statusCode >= 500)
                _logger.LogError("HTTP {Method} {Path} respondio {StatusCode} en
{ElapsedMs} ms",
                                context.Request.Method,      context.Request.Path,      statusCode,
elapsedMs);
        }
    }
}
```

```

        else if (statusCode >= 400)
            _logger.LogWarning("HTTP {Method} {Path} respondió {StatusCode} en
{ElapsedMs} ms",
                context.Request.Method,    context.Request.Path,    statusCode,
elapsedMs);
        else
            _logger.LogInformation("HTTP {Method} {Path} respondió {StatusCode}
en {ElapsedMs} ms",
                context.Request.Method,    context.Request.Path,    statusCode,
elapsedMs);
    }
}
}

```

Emite exactamente un evento por petición, con el nivel derivado del código de respuesta. Esta correspondencia entre severidad HTTP y severidad de registro permite que una consulta por nivel Error recupere de inmediato todas las fallas del servidor sin necesidad de conocer rutas ni códigos concretos. El uso de finally garantiza el registro incluso cuando la petición termina en excepción.

2.4 Verificación del comportamiento

Prueba	Resultado
Encabezado X-Correlation-ID presente en la respuesta	589679fd-3f4c-49bd-968f-39966dc3dbb5
Excepción controlada localmente, sin Search/Index	→ HTTP 200; Api/user/-999 → HTTP 400
Excepción no controlada, atendida por el middleware global	Comment/AddComment → HTTP 500 con JSON {error, correlationId}
Coincidencia entre el identificador del log y el del encabezado	Verificada línea por línea

ExceptionHandlerMiddleware.cs.]

3. P3G-4 y P3G-5 — Volumen de tráfico ejecutado

Ambas prácticas solicitan volúmenes solapados, por lo que se ejecutó una sola corrida combinada que cubre o excede los mínimos de las dos.

Categoría	P3G-4	P3G-5	Ejecutado
Página principal	—	30	32
Búsquedas válidas	100	100	100
Búsquedas vacías	—	20	20
Búsquedas con caracteres especiales	—	20	20
Búsquedas tipo SQL Injection	30	20	30
Logins válidos	50	50	53
Logins inválidos	100	100	100
Comentarios válidos	—	100	100
Comentarios con carga XSS	30	30	30
Consultas a la API	200	200	225
Excepciones controladas	—	20	22
Excepciones no controladas	—	10	12
Warnings generados	—	20	143

4. Resultados obtenidos

Nivel	Cantidad
Debug	416
Information	2760
Warning	143
Error	46

Nivel	Cantidad
Fatal	0
Total	3365

4.1 Desglose de los 46 eventos de nivel Error

Origen	Cantidad	Explicación
SearchController ApiController	/ 22	LogError local; la petición continúa y responde 200 o 400
ExceptionHandlerMiddleware	12	Excepciones no controladas que derivan en respuesta 500
RequestLoggingMiddleware	12	Un evento adicional por cada una de esas 12 respuestas 500

La duplicación de los doce últimos no constituye un defecto sino una consecuencia esperada de la instrumentación por capas: el middleware de excepciones registra la **causa** con su traza de pila, mientras el de peticiones registra el **efecto** observable por el cliente junto con la duración total. Ambos comparten el mismo identificador de correlación, de modo que la consulta los presenta unidos.

4.2 Desglose de los 143 eventos de nivel Warning

Origen	Cantidad	Explicación
AuthEvent:LoginFailed	100	Coincide exactamente con los 100 logins inválidos enviados
AuthEvent:AccesoNoAutorizado	15	5 por identificadores inválidos y 10 por recursos inexistentes
RequestLoggingMiddleware (4xx)	26	11 respuestas 400 más 15 respuestas 403
Advertencia de Entity Framework Core	1	Propiedad Balance sin tipo de columna explícito
Otros	1	—

4.3 Hallazgo derivado del análisis

Al consultar `/api/user/{id}` con identificadores inexistentes desde una sesión activa, la aplicación **no respondió 404 sino 403**, registrando `AccesoNoAutorizado`. La causa se encuentra en el orden de las verificaciones de `ApiController.GetUser`: la comprobación de pertenencia del recurso —la mitigación de IDOR implementada en la rama `secure`— se evalúa **antes** de consultar la existencia del registro, por lo que ninguna petición sobre un identificador ajeno alcanza jamás la verificación de existencia.

Este orden es el correcto desde la perspectiva de seguridad. Responder 404 ante un recurso inexistente y 403 ante uno existente pero ajeno revelaría, a un usuario sin permisos, qué identificadores están en uso, habilitando la enumeración por diferencia de respuestas. Al unificar ambos casos bajo 403, el sistema no filtra esa información.

El aspecto relevante es que este comportamiento **no era evidente en la lectura del código** y se hizo visible únicamente al revisar la distribución de códigos de respuesta en el registro, lo que ilustra el valor del logging estructurado más allá del ejercicio técnico.

5. Respuestas a las preguntas de análisis

1. Eventos Information registrados: 2760.

2. Eventos Warning registrados: 143, de los cuales 100 corresponden a `AuthEvent.LoginFailed`. Desglose completo en la sección 4.2.

3. Eventos Error registrados: 46, resultado de 22 excepciones controladas, 12 no controladas y 12 registros adicionales del middleware de peticiones sobre esas mismas respuestas 500.

4. Controlador con mayor número de registros: contabilizando por `SourceContext` y excluyendo infraestructura, `ApiController` con **567** eventos, seguido de `SearchController` (510), `AuthController` (505) y `CommentController` (396). Incluyendo la infraestructura propia, `RequestLoggingMiddleware` los supera con 764, cifra que coincide exactamente con el total de peticiones al emitir un evento por cada una.

5. Endpoint con mayor número de solicitudes: `GET /Search/Index` con **181** peticiones —100 válidas, 20 vacías, 20 con caracteres especiales, 30 de tipo SQL Injection y 11 de excepción controlada—, seguido de `POST /Auth/Login` con 159 y `POST /Comment/AddComment` con 136.

6. Dirección IP con mayor actividad: `::1`, localhost sobre IPv6, presente en 954 líneas. Es la única dirección observada, dado que la totalidad de la carga se generó

desde la misma máquina. En un entorno con tráfico externo, la misma consulta agrupada por dirección revelaría la distribución real de orígenes y permitiría detectar concentraciones anómalas.

7. Intentos de autenticación fallidos: 100, frente a **53** inicios de sesión exitosos. La relación entre ambos —cerca de dos fallos por cada éxito— constituye por sí misma un indicador vigilable: una desviación pronunciada respecto de la línea base sugiere un ataque de fuerza bruta en curso.

8. Intentos de SQL Injection identificados: 30, correspondientes exactamente a los payloads enviados a /Search/Index (' OR '1'='1, '; DROP TABLE Users;--, ' UNION SELECT..., admin'--, ' OR 1=1--, 1' AND '1'='1). Resultaron identificables porque el parámetro de búsqueda se registra íntegro, y todos fueron neutralizados por la consulta parametrizada de Entity Framework Core: **quedaron registrados pero no llegaron a ejecutarse como SQL**.

9. Posibles intentos de XSS registrados: 30, correspondientes a los comentarios con carga maliciosa (<script>, onerror=, onload=, <svg>, <iframe>). Al igual que en el caso anterior, se almacenaron como texto sin ejecutarse en el servidor. Cabe precisar que un XSS almacenado no se ejecuta en el servidor en ningún caso: su ejecución dependería de que la vista renderizara el contenido sin codificar, condición ya mitigada en la rama secure.

10. Solicitud con mayor tiempo de ejecución: POST /Auth/Login con 1707.48 ms, identificador de correlación 1fcbd2ba-b8c0-422c-85bb-69990d6d24a8. La investigación de la causa, posible gracias al desglose por correlación, muestra que la consulta a la base de datos consumió apenas 111 ms y que el tiempo restante corresponde a BCrypt.Verify. Esa lentitud es **intencional**: BCrypt es un algoritmo de hashing adaptativo cuyo factor de costo encarece deliberadamente cada verificación para degradar la viabilidad de los ataques por fuerza bruta. Se trata por tanto de un control de seguridad operando según lo previsto y no de un problema de rendimiento.

Este caso ilustra un principio relevante: la latencia elevada no equivale a defecto, y sin el detalle por componente la conclusión razonable habría sido optimizar precisamente el mecanismo que protege las credenciales.

11. ¿Fue posible localizar una petición específica usando únicamente el CorrelationId? Sí. Con la consulta CorrelationId = '1fcbd2ba-b8c0-422c-85bb-69990d6d24a8' se reconstruyó la petición completa en orden cronológico:

[10:07:15.515] [Information] Inicio Auth.Login (POST) Usuario:admin IP:::1

[10:07:15.926] [Warning] No store type was specified for the decimal property 'Balance'...

[10:07:16.928] [Information] Executed DbCommand (111ms) SELECT TOP(1) ...
FROM [Users]

WHERE [u].[Username] = @username

[10:07:16.978] [Debug] Verificando hash BCrypt para Usuario:admin
(password nunca se registra en logs)

[10:07:17.195] [Information] AuthEvent:LoginSuccess Usuario:admin IP:::1
DuracionMs:1679

[10:07:17.200] [Information] HTTP POST /Auth/Login respondio 302 en 1707.4801
ms

Seis eventos provenientes de cuatro componentes distintos —el controlador, Entity Framework Core, el registro de depuración propio y el middleware de peticiones— recuperados en orden exacto mediante una sola búsqueda, sin conocer de antemano ni la hora ni el usuario involucrado. Sobre un total de 3365 eventos, la alternativa habría sido correlacionar manualmente por marca temporal, procedimiento inviable con peticiones concurrentes.

6. Tabla resumen

Métrica	Valor
Peticiones HTTP generadas	764
Eventos de log totales	3365
Information / Warning / Error / Debug	2760 / 143 / 46 / 416
Logins exitosos / fallidos	53 / 100
Intentos de SQL Injection identificados	30 de 30
Comentarios con carga XSS identificados	30 de 30
Excepciones controladas / no controladas	22 / 12
Endpoint más solicitado	GET /Search/Index (181)

Métrica	Valor
Controlador con más registros	ApiController (567)
IP con mayor actividad	::1 (única, entorno local)
Solicitud más lenta	POST /Auth/Login, 1707.48 ms
Contraseñas detectadas en los logs	0, sobre 153 intentos de autenticación
Trazabilidad por CorrelationId	Confirmada: 6 eventos, 4 componentes, 1 consulta

7. Conclusiones

Los tres middlewares quedaron integrados en el orden necesario para que el identificador de correlación esté disponible en todo lo que se registra durante una petición, sin modificar los controladores instrumentados en la práctica anterior. La propagación automática mediante LogContext demuestra que la trazabilidad puede incorporarse como una propiedad transversal de la infraestructura y no como una obligación repetida en cada punto del código.

Se generaron y verificaron las doce categorías de prueba exigidas, con conteos que cuadran de forma exacta entre el archivo de log y la API de Seq. La capacidad de explicar cada cifra hasta el detalle —el desglose 22 + 12 + 12 que compone los 46 errores— constituye evidencia de que la instrumentación se comprende y no solo se ejecuta.

El análisis reveló un comportamiento no evidente del código: la verificación de pertenencia del recurso se evalúa antes que la de existencia, unificando bajo 403 casos que un diseño ingenuo distinguiría con 404 y habilitaría así la enumeración de identificadores. Que este orden correcto se hiciera visible únicamente a través del registro ilustra el valor del logging estructurado como instrumento de verificación de los propios controles de seguridad.

Finalmente, se confirmó sobre 153 intentos de autenticación que ninguna contraseña alcanzó los registros, y se demostró que un identificador de correlación permite reconstruir el ciclo de vida completo de una petición —incluidas las llamadas internas del ORM— con una sola consulta.